

UNIVERSIDADE CATÓLICA DO SALVADOR
CURSO DE ENGENHARIA DE SOFTWARE

DISCIPLINA: SISTEMAS DISTRIBUÍDOS

CARLOS EDUARDO SAYÃO SANTANA JUNIOR
JOAO MARCOS DE FREITAS SOUZA
MATHEUS PELTIER DE OLIVEIRA ARANHA

TÍTULO DO RELATÓRIO: JOGO XADREZ MULTIJOGADOR COM RPC

SALVADOR
2026

CARLOS EDUARDO SAYÃO SANTANA JUNIOR
JOAO MARCOS DE FREITAS SOUZA
MATHEUS PELTIER DE OLIVEIRA ARANHA

TÍTULO DO RELATÓRIO: JOGO XADREZ MULTIJOGADOR COM RPC

Relatório técnico apresentado à disciplina de Sistemas Distribuídos do Curso de Engenharia de software da Universidade Católica do Salvador, como requisito parcial para avaliação da unidade 2.

Professor(a): Adelito Tosta Inacio da Silva

SALVADOR

2026

RESUMO

Este trabalho apresenta o desenvolvimento de um jogo de xadrez multijogador em rede, elaborado na disciplina de Sistemas Distribuídos com o objetivo de aplicar conceitos de comunicação entre processos e de invocação remota. A solução adota o modelo cliente-servidor e utiliza o gRPC, implementado em Java 21 com Protocol Buffers, na comunicação entre os clientes e o servidor central. O contrato define três operações remotas: conectar jogador, enviar jogada e obter o estado do jogo. O servidor concentra o estado da partida e a validação das jogadas, enquanto os clientes capturam as ações dos jogadores e exibem o estado recebido. A arquitetura reúne o stub, no cliente, o skeleton, no servidor, e o próprio servidor gRPC como middleware, responsável por registrar o serviço. Para evitar condições de corrida em requisições simultâneas, os métodos do serviço são sincronizados, de modo que uma única chamada altere o estado da partida por vez. A atualização dos jogadores ocorre por consultas periódicas (polling) ao estado do jogo, pois o contrato não utiliza streaming. O projeto encontra-se em desenvolvimento, com a conexão dos jogadores implementada e testada e as etapas de validação das jogadas e de interface gráfica previstas.

Palavras-chave: *sistemas distribuídos; gRPC; invocação remota; concorrência; xadrez multijogador.*

LISTA DE ILUSTRAÇÕES

Figura 1 – Diagrama da arquitetura da solução……11
Figura 2 – Diagrama de sequência de uma jogada……14

LISTA DE ABREVIATURAS E SIGLAS

ABNT: Associação Brasileira de Normas Técnicas

API: Application Programming Interface

gRPC: Google Remote Procedure Call

HTTP: Hypertext Transfer Protocol

IA: Inteligência Artificial

IP: Internet Protocol

IPC: Inter-Process Communication

JSON: JavaScript Object Notation

JVM: Java Virtual Machine

Protobuf: Protocol Buffers

RMI: Remote Method Invocation

RPC: Remote Procedure Call

TCP: Transmission Control Protocol

SUMÁRIO

1 INTRODUÇÃO **7**
1.1 DESCRIÇÃO DO JOGO 7
1.2 MOTIVAÇÃO DA ESCOLHA DO PROTOCOLO 8
1.3 ORGANIZAÇÃO DO RELATÓRIO 9
2 ARQUITETURA DA SOLUÇÃO **10**
2.1 DIAGRAMA DA ARQUITETURA 11
2.2 FLUXO DE MENSAGENS A CADA JOGADA 12
3 ESPECIFICAÇÃO DAS INTERFACES REMOTAS **15**
3.1 DEFINIÇÃO DA INTERFACE 15
3.2 DESCRIÇÃO DOS MÉTODOS 15
3.3 EXCEÇÕES TRATADAS 16
3.4 OBJETOS TRANSFERIDOS 16
4 TRATAMENTO DE CONCORRÊNCIA E ESTADO DO JOGO **17**
4.1 MODELO DE EXECUÇÃO DO SERVIDOR 17
4.2 REPRESENTAÇÃO DO ESTADO DO JOGO 17
4.3 MECANISMOS DE SINCRONIZAÇÃO 17
4.4 CENÁRIOS DE CONFLITO E TESTES 17
5 MANUAL DE COMPILAÇÃO E EXECUÇÃO **18**
5.1 REQUISITOS DO AMBIENTE 18
5.2 OBTENÇÃO E COMPILAÇÃO DO CÓDIGO 18
5.3 EXECUÇÃO DO SERVIDOR 18
5.4 EXECUÇÃO DOS CLIENTES 18
5.5 PROBLEMAS COMUNS 19
6 CONCLUSÃO E AUTOAVALIAÇÃO **20**
6.1 DIFICULDADES NA IMPLEMENTAÇÃO DO BACKEND 20
6.2 RELATO DO USO DE INTELIGÊNCIA ARTIFICIAL NO FRONTEND 20
6.3 AUTOAVALIAÇÃO DO GRUPO 20
REFERÊNCIAS **21**
APÊNDICE A – CÓDIGO-FONTE DAS INTERFACES REMOTAS **22**

1 INTRODUÇÃO

O presente trabalho foi desenvolvido no âmbito da disciplina de Sistemas Distribuídos, com o objetivo de aplicar na prática conceitos relacionados à comunicação entre processos (IPC) e à comunicação remota em ambientes distribuídos. Para isso, está sendo desenvolvido um jogo de xadrez multiplayer utilizando a tecnologia gRPC (Google Remote Procedure Call), implementada na linguagem Java.

A aplicação utiliza uma arquitetura cliente-servidor, na qual um servidor central é responsável por gerenciar as partidas e processar as solicitações realizadas pelos jogadores por meio de chamadas remotas. Os clientes, por sua vez, permitem que os usuários participem das partidas e enviem suas jogadas ao servidor.

Dessa forma, o objetivo geral do projeto é desenvolver um jogo de xadrez em rede que permita a interação entre dois jogadores por meio da comunicação remota, explorando conceitos fundamentais de Sistemas Distribuídos, como comunicação entre processos, serialização de dados, gerenciamento de estado, sincronização e concorrência.

1.1 DESCRIÇÃO DO JOGO

O jogo escolhido para o desenvolvimento do projeto é o xadrez, um jogo de estratégia disputado entre dois jogadores em um tabuleiro composto por 64 casas, organizadas em 8 linhas e 8 colunas. Cada jogador controla 16 peças, sendo elas um rei, uma dama, duas torres, dois bispos, dois cavalos e oito peões. As peças possuem movimentos específicos, que determinam as possibilidades de deslocamento pelo tabuleiro.

O principal objetivo do jogo é aplicar o xeque-mate ao rei adversário, situação em que o rei está ameaçado e não possui nenhum movimento legal para escapar. A partida também pode terminar em empate, como nos casos de afogamento.

Cada jogada consiste no deslocamento de uma peça de uma posição de origem para uma posição de destino, respeitando as regras de movimentação correspondentes ao tipo da peça. Os jogadores realizam seus movimentos

alternadamente, sendo um responsável pelas peças brancas e o outro pelas peças pretas.

No projeto, o jogo será disponibilizado por meio de uma arquitetura multiplayer, permitindo que dois jogadores se conectem ao servidor central para participar de uma mesma partida. Durante a conexão, cada jogador recebe um identificador único e uma cor correspondente às peças que irá controlar. O primeiro jogador conectado recebe as peças brancas, enquanto o segundo recebe as peças pretas.

O servidor será responsável por armazenar as informações da partida, controlar os turnos, validar as jogadas e disponibilizar o estado atualizado do tabuleiro aos clientes. Dessa maneira, ambos os jogadores poderão acompanhar o andamento da partida de forma sincronizada.

1.2 MOTIVAÇÃO DA ESCOLHA DO PROTOCOLO

A escolha do gRPC como tecnologia de comunicação remota foi motivada pela sua capacidade de permitir que aplicações distribuídas realizem chamadas de procedimentos em um servidor remoto de maneira semelhante à invocação de métodos locais. Essa característica facilita a comunicação entre os clientes e o servidor, permitindo que diferentes operações do jogo sejam executadas remotamente.

Segundo Tanenbaum e Van Steen (2007), a chamada de procedimento remoto (RPC) busca simplificar a comunicação entre processos em diferentes máquinas, permitindo que operações remotas sejam realizadas de maneira semelhante às chamadas de procedimentos locais. Essa característica torna o protocolo adequado para aplicações que utilizam a arquitetura cliente-servidor, como o jogo desenvolvido neste projeto.

Diferentemente do Java RMI, que possui um modelo de comunicação orientado a objetos e é voltado principalmente para aplicações desenvolvidas em Java, o gRPC utiliza uma abordagem baseada em serviços e métodos remotos, oferecendo suporte à comunicação entre aplicações desenvolvidas em diferentes linguagens de programação. Essa característica proporciona maior flexibilidade para possíveis expansões futuras do sistema.

Outro fator determinante para a escolha foi a utilização do Protocol Buffers (Protobuf), mecanismo utilizado pelo gRPC para definir os serviços, as mensagens e os respectivos tipos de dados utilizados na comunicação. Além disso, o Protobuf realiza a serialização das mensagens em formato binário, proporcionando uma representação compacta dos dados transmitidos pela rede.

A tecnologia também disponibiliza ferramentas para geração automática dos códigos necessários à comunicação, incluindo os stubs utilizados pelos clientes para realizar as chamadas remotas. Isso facilita a integração entre as diferentes partes da aplicação e reduz a necessidade de implementar manualmente os mecanismos de comunicação.

No desenvolvimento do projeto, o gRPC permite definir operações específicas para o funcionamento do jogo, como conectar jogadores, enviar jogadas e consultar o estado da partida. Assim, o servidor concentra o gerenciamento da lógica do jogo, enquanto os clientes realizam as solicitações por meio dos serviços disponibilizados.

Portanto, a escolha do gRPC está relacionada à sua flexibilidade, ao suporte à comunicação entre diferentes plataformas, à utilização do Protocol Buffers e à geração automática dos componentes de comunicação, tornando-o adequado às necessidades do projeto e aos objetivos de aprendizagem da disciplina de Sistemas Distribuídos.

1.3 ORGANIZAÇÃO DO RELATÓRIO

Este relatório está organizado em seções que apresentam o desenvolvimento e o funcionamento da aplicação.

A Seção 1 apresenta a introdução do projeto, a descrição do jogo, a motivação para a escolha do protocolo gRPC e a organização do relatório.

A Seção 2 apresenta a arquitetura da solução, descrevendo os componentes do sistema, a comunicação entre cliente e servidor e as tecnologias utilizadas no desenvolvimento.

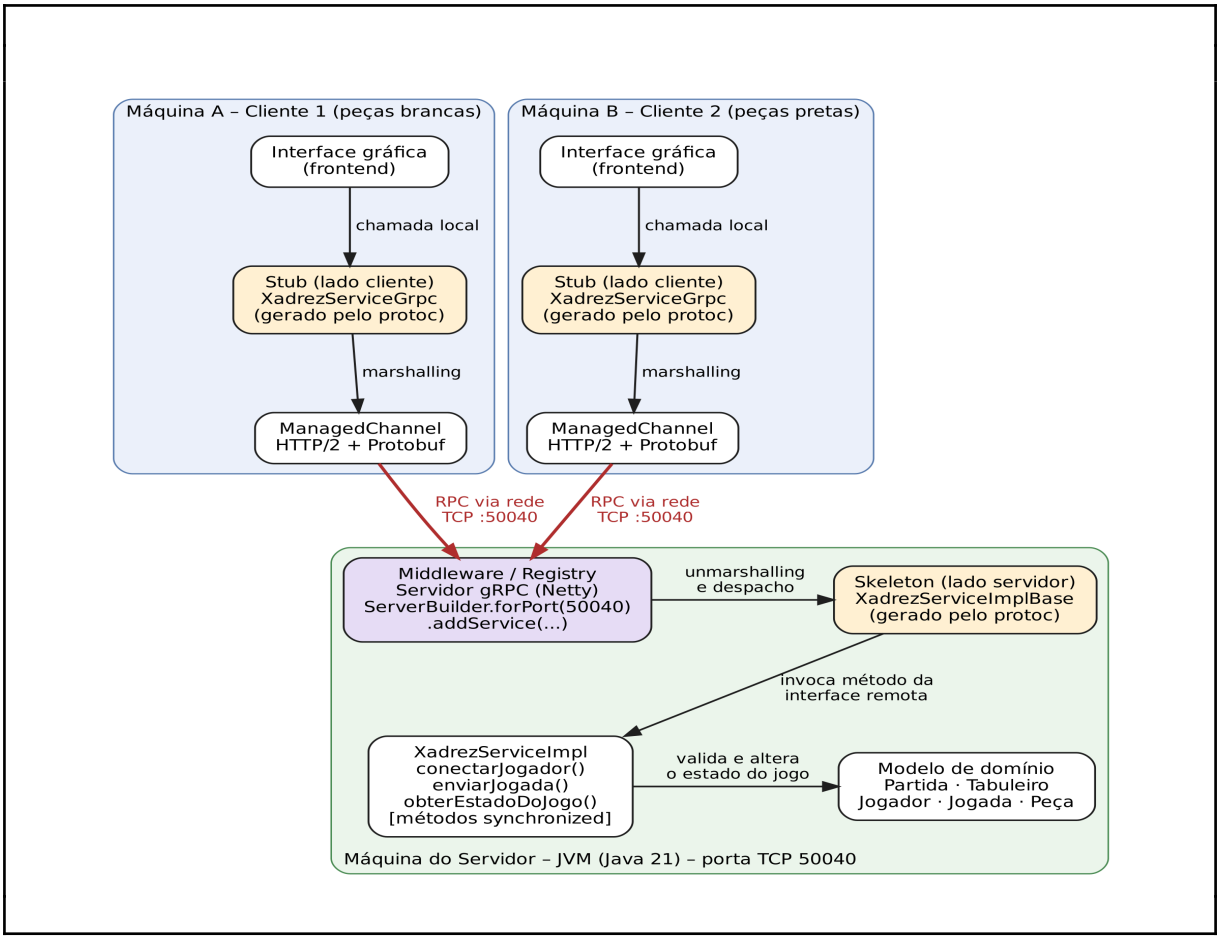
2 ARQUITETURA DA SOLUÇÃO

A solução adota o modelo cliente-servidor com comunicação remota baseada em gRPC (Remote Procedure Call). Existe um único servidor central, responsável por manter o estado da partida e validar todas as jogadas, e dois clientes, um para cada jogador, executados em processos ou máquinas distintas. Os clientes não possuem regras de jogo: apenas capturam as ações do usuário, as enviam ao servidor por chamadas remotas e exibem o estado devolvido.

O backend foi desenvolvido em Java 21, com Apache Maven para o gerenciamento das dependências e da compilação. A comunicação utiliza gRPC na versão 1.62.2 (módulos grpc-netty-shaded, grpc-protobuf, grpc-stub e grpc-services) e Protocol Buffers na versão 3.25.3 (protoc) para a definição do contrato e a serialização das mensagens. A biblioteca Gson 2.10.1 também faz parte das dependências do projeto. O frontend foi desenvolvido em Angular , conectado ao servidor por meio do stub gerado.

2.1 DIAGRAMA DA ARQUITETURA

Figura 1 – Diagrama da arquitetura da solução



Fonte: Elaborado pelos autores (2026).

Os componentes representados na Figura 1 têm as seguintes funções:

- ***Interface gráfica (frontend):*** *exibe o tabuleiro e recebe as ações do jogador, como a seleção da casa de origem e da casa de destino. Ela invoca os métodos do stub como se fossem chamadas locais. A validação das jogadas é feita exclusivamente no servidor.*
- ***Stub (lado cliente):*** *código gerado pelo compilador protoc a partir do arquivo xadrez.proto, na classe XadrezServiceGrpc. Ele representa o servidor dentro do processo do cliente, expondo os métodos conectarJogador, enviarJogada e obterEstadoDoJogo, correspondentes aos rpc ConectarJogador, EnviarJogada e ObterEstadoDoJogo do contrato. Ao ser chamado, converte os parâmetros em mensagens Protobuf (marshalling) e as entrega ao canal de comunicação.*

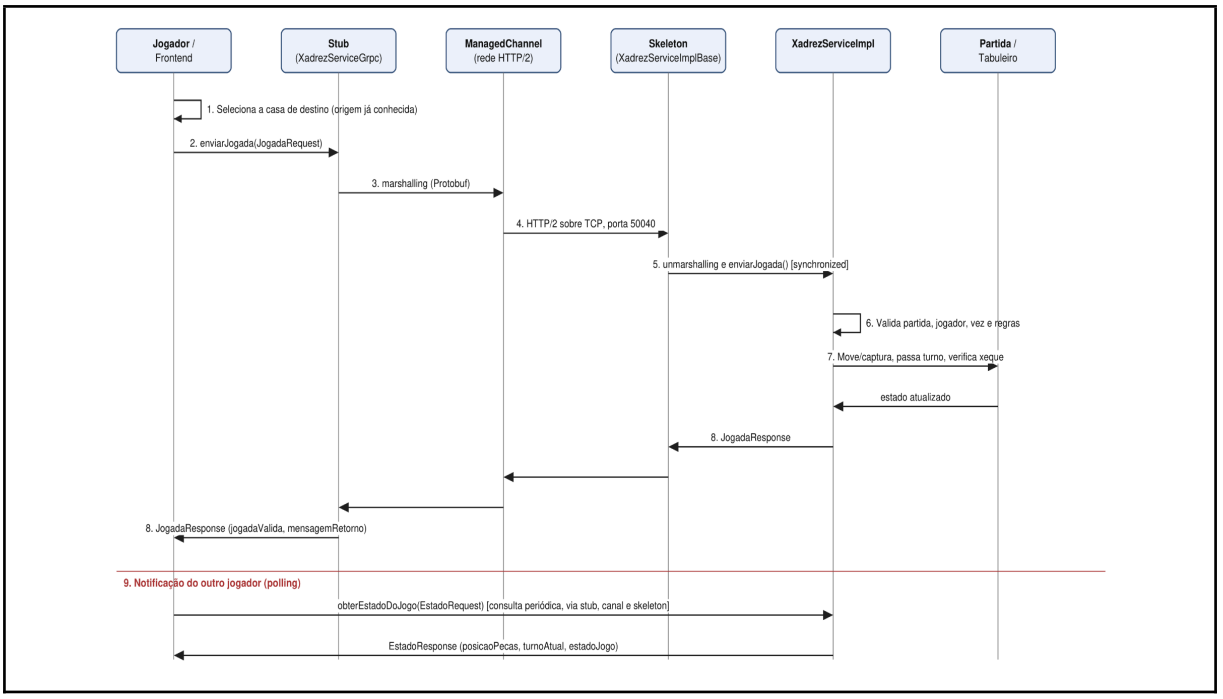
- ***ManagedChannel:*** *canal de comunicação do lado cliente. Ele mantém a conexão com o servidor, identificado pelo par endereço IP e porta, e transporta as mensagens serializadas sobre HTTP/2.*
- ***Rede:*** *a comunicação entre clientes e servidor ocorre sobre TCP, na porta 50040. Cada cliente se conecta ao endereço da máquina servidora nessa porta.*
- ***Middleware/Registry (servidor gRPC):*** *o gRPC não utiliza um serviço de nomes separado, como o rmiregistry do RMI. Esse papel é exercido pelo próprio servidor gRPC, criado com ServerBuilder.forPort(50040). O serviço é registrado nele com addService, e o servidor passa a aceitar conexões, receber as mensagens e encaminhá-las ao serviço correto. A transporte é feita pelo Netty, incluído no projeto pela dependência grpc-netty-shaded.*
- ***Skeleton (lado servidor):*** *classe abstrata gerada pelo protoc (XadrezServiceImplBase). Recebe a chamada já desserializada e a despacha para o método correspondente da implementação, que por sua vez recebe um StreamObserver para devolver a resposta ao cliente.*
- ***XadrezServiceImpl:*** *implementação do serviço remoto, escrita pela equipe. Contém os métodos invocáveis remotamente (conectarJogador, enviarJogada e obterEstadoDoJogo), declarados como synchronized para que apenas uma requisição seja processada por vez e não haja condição de corrida sobre o estado do jogo.*
- ***Modelo de domínio:*** *classes Partida, Jogador, Jogada, Posicao e Peca, que mantêm o estado da partida (jogadores, cores, turno e situação), e Tabuleiro, que representa as casas e as peças. As regras de movimento ficam em um validador de regras, classe própria usada pelo serviço antes de aplicar cada jogada. O estado existe somente no servidor.*

2.2 FLUXO DE MENSAGENS A CADA JOGADA

1. ***Ação do usuário:*** *O jogador seleciona a casa de destino (a casa de origem já é enviada a partir de onde a peça escolhida está).*
2. ***Chamada ao Stub:*** *A interface chama enviarJogada no stub XadrezServiceGrpc, passando um JogadaRequest.*

3. ***Marshalling:*** *O stub converte as informações enviadas para o formato binário do protobuf.*
4. ***Transmissão:*** *O ManagedChannel envia a mensagem por HTTP/2 sobre TCP, para a porta 50040 do servidor.*
5. ***Skeleton:*** *O Netty recebe os dados, o skeleton (XadrezServiceImplBase) desserializa e chama enviarJogada do XadrezServiceImpl. O método é synchronized, então uma jogada por vez é processada.*
6. ***Execução e Validação:***
    a. *Partida e jogador existem, caso contrário gera resposta de erro.*
    b. *A partida está em andamento e é a vez do jogador.*
    c. *As posições são válidas e convertidas para os índices do tabuleiro.*
    d. *O validador de regras confirma o movimento da peça, o lance não pode ser inválido (deixar o próprio rei em cheque)*
7. ***Atualização do estado:*** *O servidor move ou captura a peça do tabuleiro, passa o turno e então verifica se é cheque, cheque-mate ou afogamento.*
8. ***Resposta:*** *O servidor devolve um JogadaResponse com jogadaValida e mensagemRetorno. Jogada ilegal não é uma mensagem de erro e sim um retorno normal com jogadaValida = false e o motivo em mensagemRetorno.*
9. ***Notificação:*** *O contrato não utiliza streaming nem callback. Por isso, o outro cliente obtém o estado atualizado da partida por meio de consultas periódicas (polling) ao método obterEstadoDoJogo, que retorna as posições das peças, o turno atual e a situação da partida. Ao receber essa resposta, o cliente atualiza o tabuleiro e identifica se é a sua vez de jogar.*

Figura 2 – Diagrama de sequência de uma jogada



Fonte: Elaborado pelos autores (2026)

3 ESPECIFICAÇÃO DAS INTERFACES REMOTAS

[Apresente a(s) interface(s) remota(s) do sistema (por exemplo, a interface Java que estende java.rmi.Remote ou o arquivo .proto/IDL). Liste todos os métodos remotamente invocáveis, sem omissões. O Quadro 1 deve ser preenchido com uma linha por método.]

3.1 DEFINIÇÃO DA INTERFACE

[Insira o código da interface remota em fonte monoespaçada, como no exemplo abaixo, substituindo-o pelo código real do grupo.]

```
public interface JogoRemoto extends Remote {
    int entrarNaPartida(String nomeJogador) throws RemoteException,
PartidaCheiaException;
    ResultadoJogada realizarJogada(int idJogador, Jogada jogada)
        throws RemoteException, JogadaInvalidaException,
ForaDoTurnoException;
    EstadoJogo obterEstado() throws RemoteException;
}
```

3.2 DESCRIÇÃO DOS MÉTODOS

Quadro 1 – Especificação dos métodos da interface remota

Método	Parâmetros (nome: tipo)	Retorno	Exceções	Descrição
[entrarNaPartida]	*[nomeJogador: String]*	*[int]*	*[RemoteException, PartidaCheiaException]*	*[Registra o jogador e retorna seu ID]*
[realizarJogada]	*[idJogador: int; jogada: Jogada]*	*[ResultadoJogada]*	*[RemoteException, JogadaInvalidaException, ForaDoTurnoException]*	*[Valida e aplica a jogada]*
[obterEstado]	*[—]*	*[EstadoJogo]*	*[RemoteException]*	*[Retorna o estado atual]*
[...]	*[...]*	*[...]*	*[...]*	*[...]*

Fonte: Elaborado pelos autores (ANO).

3.3 EXCEÇÕES TRATADAS

[Descreva cada exceção (de comunicação e de regra de negócio), em que situação é lançada e como o cliente a trata.]

Quadro 2 – Exceções tratadas

Exceção	Condição de lançamento	Tratamento no cliente
[RemoteException]	*[Falha de comunicação com o servidor]*	*[Exibe aviso e tenta reconectar]*
[JogadaInvalidaException]	*[Jogada viola as regras]*	*[Informa o jogador e solicita nova jogada]*
[...]	*[...]*	*[...]*

Fonte: Elaborado pelos autores (ANO).

3.4 OBJETOS TRANSFERIDOS

[Descreva as classes/mensagens trafegadas entre cliente e servidor (por exemplo, Jogada, EstadoJogo), seus atributos e o mecanismo de serialização utilizado (Serializable, Protocol Buffers, JSON etc.).]

4 TRATAMENTO DE CONCORRÊNCIA E ESTADO DO JOGO

[Explique como o backend garante a consistência do estado do jogo quando múltiplos clientes efetuam requisições simultâneas.]

4.1 MODELO DE EXECUÇÃO DO SERVIDOR

[Descreva como o middleware despacha as requisições (por exemplo, o Java RMI pode executar chamadas concorrentes em threads distintas; servidores gRPC usam pools de threads). Explique o impacto disso sobre o estado compartilhado.]

4.2 REPRESENTAÇÃO DO ESTADO DO JOGO

[Descreva as estruturas de dados que armazenam o estado (tabuleiro, jogadores, turno, placar) e onde residem (memória, banco de dados).]

4.3 MECANISMOS DE SINCRONIZAÇÃO

[Detalhe os mecanismos adotados (synchronized, locks, ReentrantLock, semáforos, estruturas concorrentes, filas, controle otimista por versão etc.), as seções críticas protegidas e a justificativa de cada escolha.]

Quadro 3 – Recursos compartilhados e mecanismos de sincronização

Recurso compartilhado	Mecanismo de sincronização	Justificativa
[Tabuleiro]	*[...]*	*[...]*
[Controle de turno]	*[...]*	*[...]*
[Lista de jogadores]	*[...]*	*[...]*

Fonte: Elaborado pelos autores (ANO).

4.4 CENÁRIOS DE CONFLITO E TESTES

[Descreva situações de condição de corrida consideradas (por exemplo, dois jogadores jogando ao mesmo tempo, jogada fora do turno, desconexão no meio da partida) e como cada uma foi resolvida. Apresente evidências de teste (logs, prints ou testes automatizados com múltiplos clientes).]

5 MANUAL DE COMPILAÇÃO E EXECUÇÃO

[Escreva instruções suficientes para que uma pessoa que não participou do projeto consiga executá-lo.]

5.1 REQUISITOS DO AMBIENTE

- *[Sistema operacional testado]*
- *[Linguagem e versão (por exemplo, JDK 21)]*
- *[Ferramentas de build e dependências (Maven, Gradle, pip, npm etc.)]*
- *[Portas de rede utilizadas e configurações de firewall]*

5.2 OBTENÇÃO E COMPILAÇÃO DO CÓDIGO

[Indique o endereço do repositório e os comandos de compilação, como no exemplo:]

```
git clone https://github.com/[grupo]/[repositorio].git
cd [repositorio]
javac -d bin src/**/*.java
```

5.3 EXECUÇÃO DO SERVIDOR

[Descreva como iniciar o middleware/registry e o servidor, incluindo parâmetros como IP e porta:]

```
rmiregistry 1099 &
java -cp bin -Djava.rmi.server.hostname=[IP] servidor.ServidorJogo
```

5.4 EXECUÇÃO DOS CLIENTES

[Descreva como iniciar cada cliente (localmente e em máquinas distintas da rede) e como conectá-lo ao servidor:]

```
java -cp bin cliente.ClienteJogo [IP_DO_SERVIDOR] 1099
```

5.5 PROBLEMAS COMUNS

[Liste erros frequentes (por exemplo, ConnectException, porta em uso, hostname incorreto) e suas soluções.]

6 CONCLUSÃO E AUTOAVALIAÇÃO

[Retome o objetivo do trabalho e sintetize os resultados alcançados, confrontando-os com o que foi proposto.]

6.1 DIFICULDADES NA IMPLEMENTAÇÃO DO BACKEND

[Relate as principais dificuldades técnicas encontradas no backend (comunicação remota, serialização, concorrência, configuração de rede etc.) e como foram superadas.]

6.2 RELATO DO USO DE INTELIGÊNCIA ARTIFICIAL NO FRONTEND

[Relate de forma explicativa como a IA foi utilizada no desenvolvimento do frontend: ferramenta(s) e versão(ões); finalidade de uso; exemplos de prompts relevantes; o que foi aproveitado diretamente, o que foi adaptado e o que foi descartado; erros ou limitações observados; e como a equipe validou o código gerado. Registre a ferramenta também nas Referências.]

6.3 AUTOAVALIAÇÃO DO GRUPO

[Avalie criticamente o resultado obtido, descreva a participação de cada integrante e indique melhorias e trabalhos futuros.]

Quadro 4 – Participação dos integrantes

Integrante	Atividades desenvolvidas
[Nome 1]	*[...]*
[Nome 2]	*[...]*
[Nome 3]	*[...]*
[Nome 4]	*[...]*

Fonte: Elaborado pelos autores (ANO).

REFERÊNCIAS

[Referências em ordem alfabética, alinhadas à esquerda, espaçamento simples e separadas por uma linha em branco, conforme a NBR 6023:2018. Exemplos:]

COULOURIS, George et al. **Sistemas distribuídos**: conceitos e projeto. 5. ed. Porto Alegre: Bookman, 2013.

ORACLE. **Java Remote Method Invocation**. [S. l.]: Oracle, [20--]. Disponível em: https://docs.oracle.com/javase/. Acesso em: dia mês. ano.

TANENBAUM, Andrew S.; VAN STEEN, Maarten. **Distributed systems**. 3. ed. [S. l.]: Maarten van Steen, 2017.

[NOME DA FERRAMENTA DE IA. Versão. Empresa, ano. Disponível em: URL. Acesso em: dia mês. ano.]

APÊNDICE A – CÓDIGO-FONTE DAS INTERFACES REMOTAS

[Elemento opcional. Inclua aqui códigos extensos, logs de testes de concorrência ou capturas de tela do jogo. Cada apêndice deve ser identificado por letra maiúscula consecutiva, travessão e título.]